

Grade Automator Plus — AI Grading Flow (End to End)

1. **A submission comes in for grading.** The system loads the rubric (the questions, point values, and grading levels the teacher set up) and the student's parsed answers.
2. **Free, deterministic grading runs first (Tier 0).** For multiple-choice / true-false ("OBJECTIVE") questions, plain Python code tries to match the student's answer directly — no AI call, no cost, and it only ever claims a question when the match is unambiguous. Anything genuinely unclear is left for the AI.
3. **The answer cache is checked next (Tier 0.5).** If a student's answer text (for a given question) exactly matches an answer that's already been graded before, the prior result is reused instead of paying for another AI call. This is what guarantees two students with identical answers get identical grades — not "probably," but by construction.
4. **If everything was resolved by steps 2-3, grading stops here** — a submission that's entirely objective/cached needs no AI call at all.
5. **Whatever's left goes to the AI grader ("Grader A").** Questions are batched (up to 5 per call) and sent to the model with the rubric, the assignment context, and the student's answers (explicitly marked as untrusted input, to guard against a student trying to talk the AI into a better grade). The model is required to write its evidence quote and reasoning *before* it's allowed to state a score — this ordering is enforced by the response schema itself, not just asked for in the prompt, so the model can't pick a number first and rationalize backward.
6. **Two safety checks run on the AI's response:**
 - **Completeness** — if any requested question is missing from the response, the whole batch is retried (up to 3 times).
 - **Evidence verification** — every quote the model cites must actually appear, verbatim, in the student's real answer. A fabricated or missing quote triggers a retry; if it's still failing on the last retry, the system accepts the grade anyway rather than losing it, but marks it as "evidence unverified" for visibility.
7. **The AI's own math is never trusted.** Every score is clamped to the question's point cap and snapped to the nearest valid rubric level in plain code — regardless of what the AI's own total says. The submission's total score and percentage are recomputed from scratch server-side.
8. **The system decides whether a second, independent opinion is needed.** This doesn't happen for every question — it's triggered selectively:
 - The whole run's confidence was low, OR
 - The AI flagged the question itself (e.g. suspected plagiarism, off-topic answer), OR
 - The AI said this particular answer was a genuine "borderline" call between two rubric levels, OR
 - The question is worth a lot of points, OR
 - **(New)** the question is an essay or short-answer question — these get a second opinion by default, regardless of the above, since subjective/judged questions are where independent graders diverge the most, OR
 - It was randomly selected as part of an ongoing 5% quality-audit sample.

9. **If triggered, a second AI model re-grades — blind.** A *different* model (never the same one that graded pass A) re-grades only the triggered questions, seeing nothing about grader A's score or reasoning. This keeps the second opinion genuinely independent.
 10. **The two opinions are compared.** If they agree, nothing changes — silently. If they disagree, grader A's score still stands (a second opinion never overwrites a number), but the submission is flagged `needs_review` with a severity level (critical / moderate / borderline) based on how far apart the two graders landed.
 11. **Fresh evaluations get written back to the answer cache** — but only if they weren't disputed by a second opinion. A disagreed-upon grade is never propagated to future students via the cache.
 12. **The grade is saved to the student's submission record.** Score, feedback, confidence, and (if applicable) the review flag are all persisted. If the assignment isn't published yet, the student doesn't see anything until the teacher releases it.
 13. **A teacher can review and resolve anything sitting in the `needs_review` queue**, via one of two endpoints:
 - **mark-reviewed** — "I looked at both graders' rationales, grader A's score stands." No score changes, no credits consumed. Clears `needs_review` and appends a `{"resolved": "confirmed", ...}` entry to `review_reasons`.
 - **update-grade** — actually changes the score. Requires the teacher to have AI credits available, because it does more than update a number: it validates the new score against the rubric's max, recomputes the percentage, marks the submission as `was_regraded`, resolves the review (`{"resolved": "overridden", ...}`), notifies the student if the grade was already published — **and then kicks off a background AI call** that regenerates the student-facing "formatted grade" write-up to reflect the corrected score. That last step is a real, billed AI call, which is why the endpoint is credit-gated even though the score change itself is a plain database write.
 14. **Separately, a nightly (free) and weekly (billed) benchmark run** against a set of synthetic students with known-correct answers, to catch any drift in grading accuracy over time — this runs independently of live grading, not as part of any individual submission's flow.
-

Flow chart





